```python
# Implementation of Ensemble Learning using Random Forest
# and Comparison with Decision Tree Classifier

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd


print("SHRAWANI KHANDAGALE.")
print("Roll no.- 69\n")
# 1. Load Dataset
iris = load_iris()
X = iris.data
y = iris.target

# 2. Split Dataset into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Create and Train Decision Tree Classifier
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# 4. Create and Train Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# 5. Make Predictions
dt_pred = dt_model.predict(X_test)
rf_pred = rf_model.predict(X_test)

# 6. Calculate Accuracy
dt_accuracy = accuracy_score(y_test, dt_pred)
rf_accuracy = accuracy_score(y_test, rf_pred)

print("Decision Tree Accuracy:", round(dt_accuracy * 100, 2), "%")
print("Random Forest Accuracy:", round(rf_accuracy * 100, 2), "%")

# 7. Compare Model Performance
results = pd.DataFrame({
    "Model": ["Decision Tree", "Random Forest"],
    "Accuracy (%)": [dt_accuracy * 100, rf_accuracy * 100]
})
```

```
print("\nModel Comparison:")
print(results.to_string(index=False))

# 8. Display Classification Reports
print("\nDecision Tree Classification Report:")
print(classification_report(y_test, dt_pred,
target_names=iris.target_names))

print("\nRandom Forest Classification Report:")
print(classification_report(y_test, rf_pred,
target_names=iris.target_names))

# 9. Plot Accuracy Comparison
plt.figure(figsize=(7, 5))
plt.bar(results["Model"], results["Accuracy (%)"])
plt.title("Decision Tree vs Random Forest Accuracy")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 110)
plt.show()

# 10. Display Confusion Matrices
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.heatmap(confusion_matrix(y_test, dt_pred), annot=True, fmt="d",
            cmap="Blues", ax=axes[0])
axes[0].set_title("Decision Tree Confusion Matrix")
axes[0].set_xlabel("Predicted")
axes[0].set_ylabel("Actual")

sns.heatmap(confusion_matrix(y_test, rf_pred), annot=True, fmt="d",
            cmap="Greens", ax=axes[1])
axes[1].set_title("Random Forest Confusion Matrix")
axes[1].set_xlabel("Predicted")
axes[1].set_ylabel("Actual")

plt.tight_layout()
plt.show()

# 11. Conclusion
print("\nConclusion:")
if rf_accuracy > dt_accuracy:
    print("Random Forest performs better than Decision Tree on this
test set.")
elif rf_accuracy < dt_accuracy:
    print("Decision Tree performs better than Random Forest on this
test set.")
else:
    print("Both models have the same accuracy on this test set.")
```

```
SHRAWANI KHANDAGALE.
Roll no.- 69

Decision Tree Accuracy: 100.0 %
Random Forest Accuracy: 100.0 %

Model Comparison:
               Model  Accuracy (%)
Decision Tree         100.0
Random Forest         100.0

Decision Tree Classification Report:
              precision    recall  f1-score   support

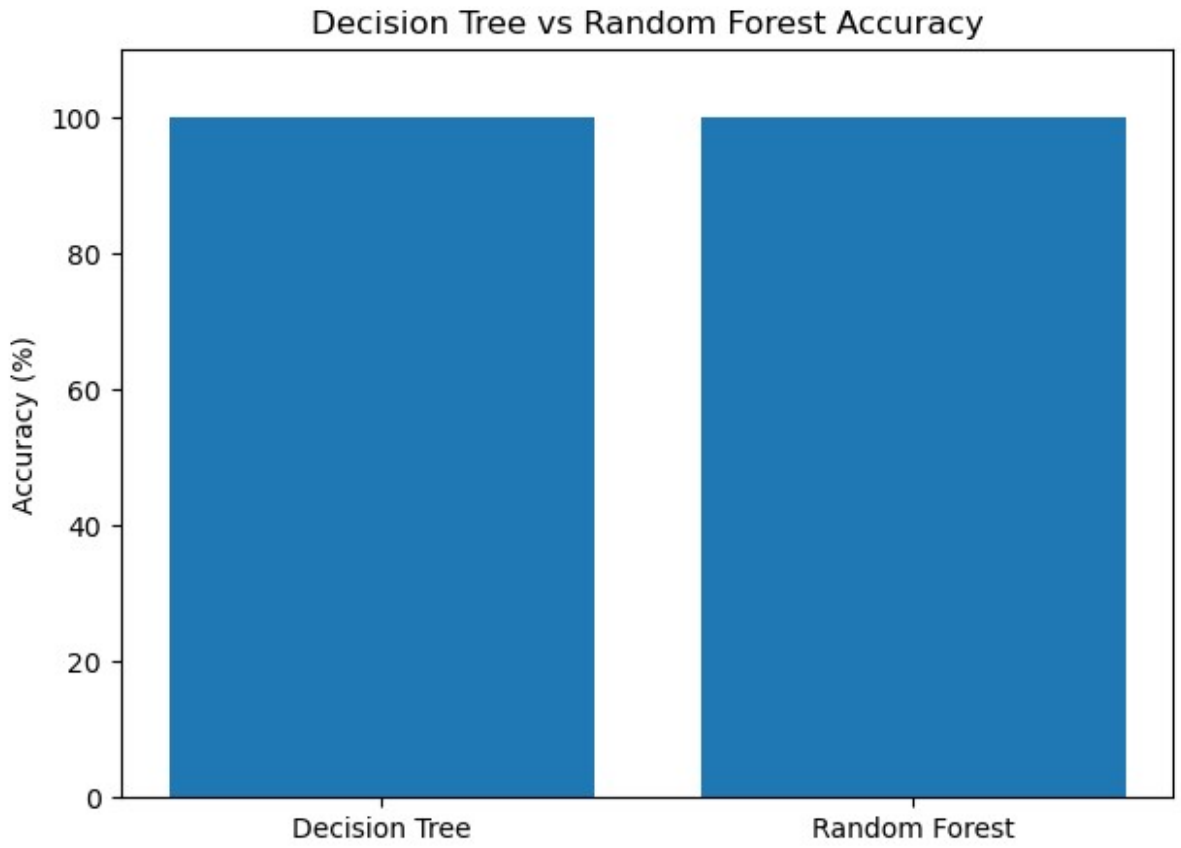
      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

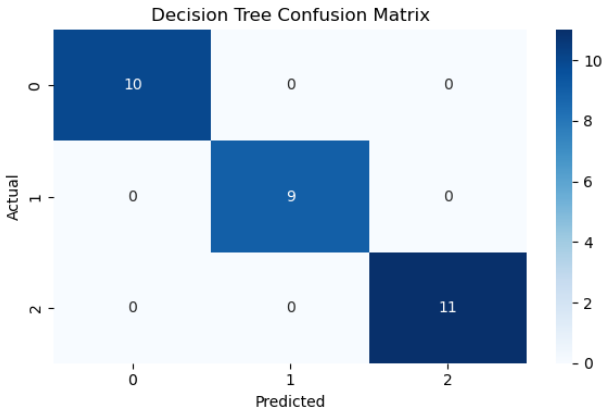
    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30

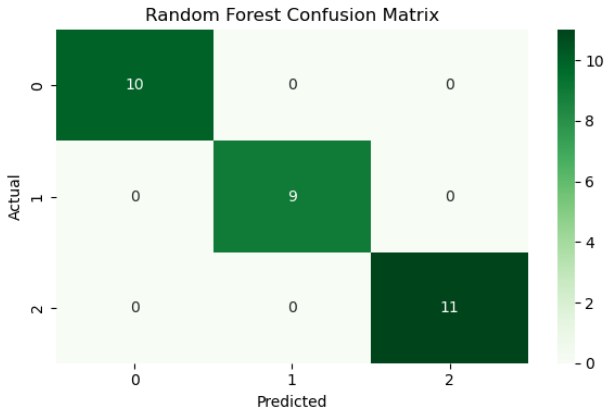

Random Forest Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```








```
Conclusion:
Both models have the same accuracy on this test set.
```